Lab 5: A bigger State Machine

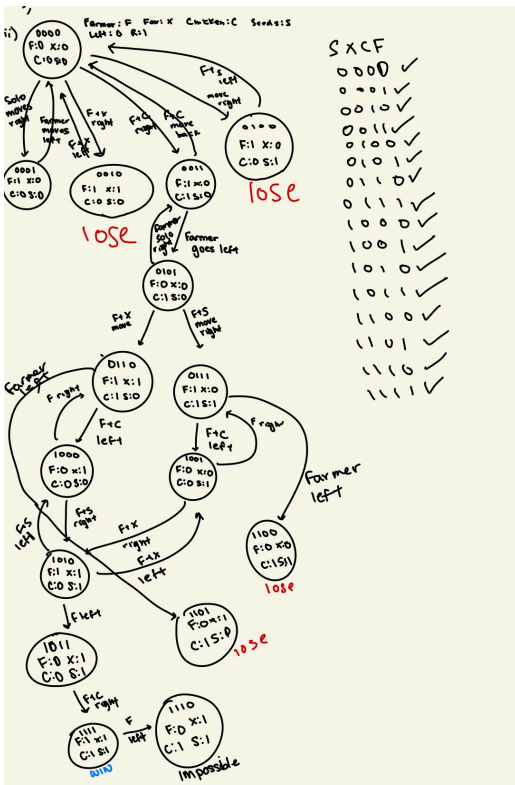
ECEN 127L: Tuesday 5:15 - 8:00 pm

Ryan Kiniris and Dylan Thornburg

Introduction

In this lab we are going to design, implement, and verify a synchronous finite 16 state machine (FSM) that models the *Farmer, Fox, Chicken, and Seeds* puzzle. The FSM looks at the four entities that are each represented by one bit of a 4-bit state vector, which shows whether they are on the left or right bank of the river. Player inputs determine how the farmer moves, either by itself or carrying one of the other entities, and the circuit outputs signals for win, lose, or impossible depending on the resulting state. The design uses Moore outputs for both the *win* and *lose* and a Mealy output for *impossible*, demonstrating both styles within one controller.

State Machine Drawing from prelab:



Procedure

We had started out by defining the module interface with clk, rst, a 3-bit input for the five move (no move, move solo, or move for each entity) options, and outputs for win, lose, and impossible. The 4-bit state register {F,X,C,S} encodes the positions of all entities and resets to 0000. We then created a 16-state diagram showing all valid transitions, self-loops for invalid moves, and terminal states for win and lose.

The FSM was implemented in Verilog using synchronous state logic and combinational next-state and output logic. A testbench was written to simulate valid and invalid gameplay,

confirm correct win/lose detection, and verify that impossible moves held the current state, as well as resets and to see if it moves after a win or lose. The final simulation confirmed proper operation across all conditions.

Results

- **What problems did you encounter while testing your steps yourself?**

  The only problems we had dealt with were timing. The monitor statement only displays when changes to inputs are made, so it was hard sometimes to see what was happening when values didn't change for repeated test cases.

- **Did any problems arise when demonstrating for the TA? What were they? Explain your thoughts on how/why these test cases escaped your own testing.**

  For the most part there were no issues. But when demoing we had to add some more no move states after a reset at the end.

SIMULATION RESULTS

State Machine test bench:

```
[dthornburg@linux402604 Lab5]$ ./simv -no_save
Chronologic VCS simulator copyright 1991-2023
Contains Synopsys proprietary information.
Compiler version V-2023.12-SP1-1_Full64; Runtime version V-2023.12-SP1-1_Full64;  Oct 28 17:47 2025
Start of the game test
Reset
state= 0, rst=1, farmer=0, farmer_fox=0, farmer_chicken=0, farmer_seeds=0, win=0, lose=0, impossible= 0, @                  0
state= 0, rst=0, farmer=0, farmer_fox=0, farmer_chicken=0, farmer_seeds=0, win=0, lose=0, impossible= 0, @                 10
End reset


Testing winning sequence 1
state=10, rst=0, farmer=0, farmer_fox=0, farmer_chicken=1, farmer_seeds=0, win=0, lose=0, impossible= 0, @                 20
state= 2, rst=0, farmer=1, farmer_fox=0, farmer_chicken=0, farmer_seeds=0, win=0, lose=0, impossible= 0, @                 30
state=14, rst=0, farmer=0, farmer_fox=1, farmer_chicken=0, farmer_seeds=0, win=0, lose=0, impossible= 0, @                 40
state= 4, rst=0, farmer=0, farmer_fox=0, farmer_chicken=1, farmer_seeds=0, win=0, lose=0, impossible= 0, @                 50
state=13, rst=0, farmer=0, farmer_fox=0, farmer_chicken=0, farmer_seeds=1, win=0, lose=0, impossible= 0, @                 60
state= 5, rst=0, farmer=1, farmer_fox=0, farmer_chicken=0, farmer_seeds=0, win=0, lose=0, impossible= 0, @                 70
state=15, rst=0, farmer=0, farmer_fox=0, farmer_chicken=1, farmer_seeds=0, win=1, lose=0, impossible= 0, @                 80
END win


Reset
state= 0, rst=1, farmer=0, farmer_fox=0, farmer_chicken=0, farmer_seeds=0, win=0, lose=0, impossible= 0, @                 90
state= 0, rst=0, farmer=0, farmer_fox=0, farmer_chicken=0, farmer_seeds=0, win=0, lose=0, impossible= 0, @                100
End reset


Testing winning sequence 2
state=10, rst=0, farmer=0, farmer_fox=0, farmer_chicken=1, farmer_seeds=0, win=0, lose=0, impossible= 0, @                110
state= 2, rst=0, farmer=1, farmer_fox=0, farmer_chicken=0, farmer_seeds=0, win=0, lose=0, impossible= 0, @                120
state=11, rst=0, farmer=0, farmer_fox=0, farmer_chicken=0, farmer_seeds=1, win=0, lose=0, impossible= 0, @                130
state= 1, rst=0, farmer=0, farmer_fox=0, farmer_chicken=1, farmer_seeds=0, win=0, lose=0, impossible= 0, @                140
state=13, rst=0, farmer=0, farmer_fox=1, farmer_chicken=0, farmer_seeds=0, win=0, lose=0, impossible= 0, @                150
state= 5, rst=0, farmer=1, farmer_fox=0, farmer_chicken=0, farmer_seeds=0, win=0, lose=0, impossible= 0, @                160
state=15, rst=0, farmer=0, farmer_fox=0, farmer_chicken=1, farmer_seeds=0, win=1, lose=0, impossible= 0, @                170
END win. Testing holding the win now.
state=15, rst=0, farmer=1, farmer_fox=0, farmer_chicken=0, farmer_seeds=0, win=1, lose=0, impossible= 0, @                180
state=15, rst=0, farmer=0, farmer_fox=1, farmer_chicken=0, farmer_seeds=0, win=1, lose=0, impossible= 0, @                190
state=15, rst=0, farmer=0, farmer_fox=0, farmer_chicken=1, farmer_seeds=0, win=1, lose=0, impossible= 0, @                200
state=15, rst=0, farmer=0, farmer_fox=0, farmer_chicken=0, farmer_seeds=1, win=1, lose=0, impossible= 0, @                210


reset
state= 0, rst=1, farmer=0, farmer_fox=0, farmer_chicken=0, farmer_seeds=0, win=0, lose=0, impossible= 0, @                220
end reset. Testing holding loss.
state= 8, rst=0, farmer=1, farmer_fox=0, farmer_chicken=0, farmer_seeds=0, win=0, lose=1, impossible= 0, @                230
Loss state 1000
state= 8, rst=0, farmer=0, farmer_fox=1, farmer_chicken=0, farmer_seeds=0, win=0, lose=1, impossible= 0, @                250
state= 8, rst=0, farmer=0, farmer_fox=0, farmer_chicken=1, farmer_seeds=0, win=0, lose=1, impossible= 0, @                260
state= 8, rst=0, farmer=0, farmer_fox=0, farmer_chicken=0, farmer_seeds=1, win=0, lose=1, impossible= 0, @                270
```

```
Reset
state= 0, rst=1, farmer=0, farmer_fox=0, farmer_chicken=0, farmer_seeds=0, win=0, lose=0, impossible= 0, @                280
Testing impossible moves
state=10, rst=0, farmer=0, farmer_fox=0, farmer_chicken=1, farmer_seeds=0, win=0, lose=0, impossible= 0, @                290
state=10, rst=0, farmer=0, farmer_fox=0, farmer_chicken=0, farmer_seeds=1, win=0, lose=0, impossible= 1, @                300
state=10, rst=0, farmer=0, farmer_fox=1, farmer_chicken=0, farmer_seeds=0, win=0, lose=0, impossible= 1, @                310


reset
state= 0, rst=1, farmer=0, farmer_fox=0, farmer_chicken=0, farmer_seeds=0, win=0, lose=0, impossible= 0, @                320


Begin Testing rest of losses now
state=12, rst=0, farmer=0, farmer_fox=1, farmer_chicken=0, farmer_seeds=0, win=0, lose=1, impossible= 0, @                330
Loss state 1100
state= 0, rst=1, farmer=0, farmer_fox=0, farmer_chicken=0, farmer_seeds=0, win=0, lose=0, impossible= 0, @                340
state= 9, rst=0, farmer=0, farmer_fox=0, farmer_chicken=0, farmer_seeds=1, win=0, lose=1, impossible= 0, @                350
Loss state 1001
state= 0, rst=1, farmer=0, farmer_fox=0, farmer_chicken=0, farmer_seeds=0, win=0, lose=0, impossible= 0, @                360
state=10, rst=0, farmer=0, farmer_fox=0, farmer_chicken=1, farmer_seeds=0, win=0, lose=0, impossible= 0, @                370
state= 2, rst=0, farmer=1, farmer_fox=0, farmer_chicken=0, farmer_seeds=0, win=0, lose=0, impossible= 0, @                380
state=14, rst=0, farmer=0, farmer_fox=1, farmer_chicken=0, farmer_seeds=0, win=0, lose=0, impossible= 0, @                390
state= 6, rst=0, farmer=1, farmer_fox=0, farmer_chicken=0, farmer_seeds=0, win=0, lose=1, impossible= 0, @                400
Loss state 0110
state= 0, rst=1, farmer=0, farmer_fox=0, farmer_chicken=0, farmer_seeds=0, win=0, lose=0, impossible= 0, @                410
state=10, rst=0, farmer=0, farmer_fox=0, farmer_chicken=1, farmer_seeds=0, win=0, lose=0, impossible= 0, @                420
state= 2, rst=0, farmer=1, farmer_fox=0, farmer_chicken=0, farmer_seeds=0, win=0, lose=0, impossible= 0, @                430
state=11, rst=0, farmer=0, farmer_fox=0, farmer_chicken=0, farmer_seeds=1, win=0, lose=0, impossible= 0, @                440
state= 3, rst=0, farmer=1, farmer_fox=0, farmer_chicken=0, farmer_seeds=0, win=0, lose=1, impossible= 0, @                450
Loss state 0011
state= 0, rst=1, farmer=0, farmer_fox=0, farmer_chicken=0, farmer_seeds=0, win=0, lose=0, impossible= 0, @                460
state= 0, rst=0, farmer=0, farmer_fox=0, farmer_chicken=0, farmer_seeds=0, win=0, lose=0, impossible= 0, clk=1 @                470
state= 0, rst=0, farmer=0, farmer_fox=0, farmer_chicken=0, farmer_seeds=0, win=0, lose=0, impossible= 0, clk=0 @                475
state= 0, rst=0, farmer=0, farmer_fox=0, farmer_chicken=0, farmer_seeds=0, win=0, lose=0, impossible= 0, clk=1 @                480
state= 0, rst=0, farmer=0, farmer_fox=0, farmer_chicken=0, farmer_seeds=0, win=0, lose=0, impossible= 0, clk=0 @                485
state= 0, rst=0, farmer=0, farmer_fox=0, farmer_chicken=0, farmer_seeds=0, win=0, lose=0, impossible= 0, clk=1 @                490
state= 0, rst=0, farmer=0, farmer_fox=0, farmer_chicken=0, farmer_seeds=0, win=0, lose=0, impossible= 0, clk=0 @                495
state= 0, rst=0, farmer=0, farmer_fox=0, farmer_chicken=0, farmer_seeds=0, win=0, lose=0, impossible= 0, clk=1 @                500
state= 0, rst=0, farmer=0, farmer_fox=0, farmer_chicken=0, farmer_seeds=0, win=0, lose=0, impossible= 0, clk=0 @                505
End of game Test
$finish called from file "tb.v", line 170.
$finish at simulation time                 510
           V C S   S i m u l a t i o n   R e p o r t
Time: 510
CPU Time:      0.200 seconds;       Data structure size:   0.0Mb
Tue Oct 28 17:47:24 2025
[dthornburg@linux402604 Lab5]$
```

Conclusion

Overall, in this lab we created, designed, and implemented a FSM state machine with sixteen states with both mealy and moore outputs, with both the win and lose being moore and impossible being the mealy. We tested all both win sequences and well as our five loss sequences and our one impossible. In the end, we had added an extra no move case to see what would happen when it doesn't move at any of the holds. This state machine could be used to simulate the game puzzle where the winning point is to get them all together on the right and to lose if either the chicken and seeds are together or fox or chicken together (both without the farmer. Because the 0111 state was impossible to ever get to, it was completely left out of the SM code.

Appendix

CODE

```
module game(
	input clk, rst, farmer, farmer_fox, farmer_chicken, farmer_seeds,
	output reg [3:0] state,
	output reg win, lose, impossible
);
	reg [3:0] curr_state, next_state;
	always @(posedge clk) begin
		if (rst) curr_state <= 4'b0000;
		else curr_state <= next_state;
	end

	always @(*) begin
		// Default outputs
		win=0;
		lose=0;
		impossible=0;

		// default next state
		next_state = curr_state;
		state = curr_state;
		case (curr_state)

			4'b0000: begin
				if (farmer) next_state = 4'b1000; //lose
				else if (farmer_fox) next_state = 4'b1100; //lose
				else if (farmer_chicken) next_state = 4'b1010;
				else if (farmer_seeds) next_state = 4'b1001; //lose
				else next_state = 4'b0000;
			end

			4'b1010: begin
				if (farmer) next_state = 4'b0010;
				else if (farmer_fox) impossible = 1;
				else if (farmer_chicken) next_state = 4'b0000;
				else if (farmer_seeds) impossible = 1;
				else next_state = 4'b1010;
			end

			4'b0010: begin
				if (farmer) next_state = 4'b1010;
				else if (farmer_fox) next_state = 4'b1110;
				else if (farmer_chicken) impossible = 1;
				else if (farmer_seeds) next_state = 4'b1011;
				else next_state = 4'b0010;
			end

			4'b1110: begin
				if (farmer) next_state = 4'b0110; //lose
				else if (farmer_fox) next_state = 4'b0010;
				else if (farmer_chicken) next_state = 4'b0100;
```

```
                        else if (farmer_seeds) impossible = 1;
                        else next_state = 4'b1110;
                end

                4'b1011: begin
                        if (farmer) next_state = 4'b0011; //lose
                        else if (farmer_fox) impossible = 1;
                        else if (farmer_chicken) next_state = 4'b0001;
                        else if (farmer_seeds) next_state = 4'b0010;
                        else next_state = 4'b1011;
                end

                4'b0100: begin
                        if (farmer) next_state = 4'b1100; //lose
                        else if (farmer_fox) impossible = 1;
                        else if (farmer_chicken) next_state = 4'b1110;
                        else if (farmer_seeds) next_state = 4'b1101;
                        else next_state = 4'b0100;
                end

                4'b0001: begin
                        if (farmer) next_state = 4'b1001; //lose
                        else if (farmer_fox) next_state = 4'b1101;
                        else if (farmer_chicken) next_state = 4'b1011;
                        else if (farmer_seeds) impossible = 1;
                        else next_state = 4'b0001;
                end

                4'b1101: begin
                        if (farmer) next_state = 4'b0101;
                        else if (farmer_fox) next_state = 4'b0001;
                        else if (farmer_chicken) impossible = 1;
                        else if (farmer_seeds) next_state = 4'b0100;
                        else next_state = 4'b1101;
                end

                4'b0101: begin
                        if (farmer) next_state = 4'b1101;
                        else if (farmer_fox) impossible = 1;
                        else if (farmer_chicken) next_state = 4'b1111;
                        else if (farmer_seeds) impossible = 1;
                        else next_state = 4'b0101;
                end

                4'b1111: begin
                        next_state = curr_state;
                end

                default: begin
                        next_state = curr_state;
                end
        endcase
        win  = (next_state == 4'b1111);
        lose = (next_state == 4'b1000 || next_state == 4'b1100  || next_state == 4'b1001  ||
next_state == 4'b0110  || next_state == 4'b0011);
```

```
        end
endmodule
```

```
module testbench();

reg clk, rst, farmer, farmer_fox, farmer_chicken, farmer_seeds;
wire [3:0] state;
wire win, lose, impossible;

game dut(clk, rst, farmer, farmer_fox, farmer_chicken, farmer_seeds, state, win, lose,
impossible);

initial clk=1;
initial rst=0;
always #5 clk = ~clk;

initial begin
$display("Start of the game test");
$monitor("state=%d, rst=%d, farmer=%d, farmer_fox=%d, farmer_chicken=%d,
farmer_seeds=%d, win=%d, lose=%d, impossible= %d, @%d", state, rst, farmer, farmer_fox,
farmer_chicken, farmer_seeds, win, lose, impossible, $time); //always displays at posedge
of clk

$display("Reset");
farmer=0; farmer_fox=0; farmer_chicken=0; farmer_seeds=0; rst=1;
#10
farmer=0; farmer_fox=0; farmer_chicken=0; farmer_seeds=0; rst=0;
#10
$display("End reset");

$display("\n");

//winning seq
$display("Testing winning sequence 1");
farmer=0; farmer_fox=0; farmer_chicken=1; farmer_seeds=0;
#10;
farmer=1; farmer_fox=0; farmer_chicken=0; farmer_seeds=0;
#10;
farmer=0; farmer_fox=1; farmer_chicken=0; farmer_seeds=0;
#10;
farmer=0; farmer_fox=0; farmer_chicken=1; farmer_seeds=0;
#10;
farmer=0; farmer_fox=0; farmer_chicken=0; farmer_seeds=1;
#10;
farmer=1; farmer_fox=0; farmer_chicken=0; farmer_seeds=0;
#10;
```

```
farmer=0; farmer_fox=0; farmer_chicken=1; farmer_seeds=0;
#10;
$display("END win");

$display("\n");

$display("Reset");
farmer=0; farmer_fox=0; farmer_chicken=0; farmer_seeds=0; rst=1;
#10
farmer=0; farmer_fox=0; farmer_chicken=0; farmer_seeds=0; rst=0;
#10
$display("End reset");

$display("\n");

//winning seq2
$display("Testing winning sequence 2");
farmer=0; farmer_fox=0; farmer_chicken=1; farmer_seeds=0;
#10;
farmer=1; farmer_fox=0; farmer_chicken=0; farmer_seeds=0;
#10;
farmer=0; farmer_fox=0; farmer_chicken=0; farmer_seeds=1;
#10;
farmer=0; farmer_fox=0; farmer_chicken=1; farmer_seeds=0;
#10;
farmer=0; farmer_fox=1; farmer_chicken=0; farmer_seeds=0;
#10;
farmer=1; farmer_fox=0; farmer_chicken=0; farmer_seeds=0;
#10;
farmer=0; farmer_fox=0; farmer_chicken=1; farmer_seeds=0;
#10;
$display("END win. Testing holding the win now.");

//test holding win
farmer=1; farmer_fox=0; farmer_chicken=0; farmer_seeds=0;
#10;
farmer=0; farmer_fox=1; farmer_chicken=0; farmer_seeds=0;
#10;
farmer=0; farmer_fox=0; farmer_chicken=1; farmer_seeds=0;
#10;
farmer=0; farmer_fox=0; farmer_chicken=0; farmer_seeds=1;
#10;

$display("\n");
$display("reset");
farmer=0; farmer_fox=0; farmer_chicken=0; farmer_seeds=0; rst=1;
#10
$display("end reset. Testing holding loss.");
farmer=1; farmer_fox=0; farmer_chicken=0; farmer_seeds=0; rst=0;
#10
```

```
$display("Loss state 1000");
//hold loss
farmer=1; farmer_fox=0; farmer_chicken=0; farmer_seeds=0;
#10;
farmer=0; farmer_fox=1; farmer_chicken=0; farmer_seeds=0;
#10;
farmer=0; farmer_fox=0; farmer_chicken=1; farmer_seeds=0;
#10;
farmer=0; farmer_fox=0; farmer_chicken=0; farmer_seeds=1;
#10;

$display("\n");
$display("Reset");
farmer=0; farmer_fox=0; farmer_chicken=0; farmer_seeds=0; rst=1;
#10
$display("Testing impossible moves");
farmer=0; farmer_fox=0; farmer_chicken=1; farmer_seeds=0; rst=0;
#10
//test impossibles
farmer=0; farmer_fox=0; farmer_chicken=0; farmer_seeds=1;
#10
farmer=0; farmer_fox=1; farmer_chicken=0; farmer_seeds=0;
#10

$display("\n");
$display("reset");
farmer=0; farmer_fox=0; farmer_chicken=0; farmer_seeds=0; rst=1;
#10

$display("\n");
$display("Begin Testing rest of losses now");
farmer=0; farmer_fox=1; farmer_chicken=0; farmer_seeds=0; rst=0;
#10
$display("Loss state 1100");
farmer=0; farmer_fox=0; farmer_chicken=0; farmer_seeds=0; rst=1;
#10

farmer=0; farmer_fox=0; farmer_chicken=0; farmer_seeds=1; rst=0;
#10
$display("Loss state 1001");
farmer=0; farmer_fox=0; farmer_chicken=0; farmer_seeds=0; rst=1;
#10

farmer=0; farmer_fox=0; farmer_chicken=1; farmer_seeds=0; rst=0;
#10;
farmer=1; farmer_fox=0; farmer_chicken=0; farmer_seeds=0;
#10;
farmer=0; farmer_fox=1; farmer_chicken=0; farmer_seeds=0;
#10;
farmer=1; farmer_fox=0; farmer_chicken=0; farmer_seeds=0;
```

```
#10;
$display("Loss state 0110");

farmer=0; farmer_fox=0; farmer_chicken=0; farmer_seeds=0; rst=1;
#10

farmer=0; farmer_fox=0; farmer_chicken=1; farmer_seeds=0; rst=0;
#10;
farmer=1; farmer_fox=0; farmer_chicken=0; farmer_seeds=0;
#10;
farmer=0; farmer_fox=0; farmer_chicken=0; farmer_seeds=1;
#10;
farmer=1; farmer_fox=0; farmer_chicken=0; farmer_seeds=0;
#10;
$display("Loss state 0011");

farmer=0; farmer_fox=0; farmer_chicken=0; farmer_seeds=0; rst=1;
#10

$monitor("state=%d, rst=%d, farmer=%d, farmer_fox=%d, farmer_chicken=%d,
farmer_seeds=%d, win=%d, lose=%d, impossible= %d, clk=%d @%d", state, rst, farmer,
farmer_fox, farmer_chicken, farmer_seeds, win, lose, impossible, clk, $time);
farmer=0; farmer_fox=0; farmer_chicken=0; farmer_seeds=0; rst=0;
#10
farmer=0; farmer_fox=0; farmer_chicken=0; farmer_seeds=0;
#10
farmer=0; farmer_fox=0; farmer_chicken=0; farmer_seeds=0;
#10
farmer=0; farmer_fox=0; farmer_chicken=0; farmer_seeds=0;
#10

$display("End of game Test");
$finish;

end

endmodule
```